

Escalabilidad y Rendimiento de Sistemas LLM

Módulo 9 · Infraestructura y Despliegue

KEDA autoscaling, warm pools, scale-to-zero, semantic caching, priorización y dimensionamiento del pool.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Escalabilidad y Rendimiento de Sistemas LLM

Escalar un sistema LLM de 100 usuarios a 10.000 no es simplemente multiplicar los recursos por 100. Los LLMs tienen características de escalabilidad únicas que hacen que los enfoques de escalabilidad estándar de aplicaciones web no funcionen directamente: el tiempo de arranque de un pod de inferencia (1-5 minutos mientras se carga el modelo en VRAM) es incompatible con el autoscaling tradicional, el KV cache en memoria es intrínsecamente stateful, y los picos de demanda de sistemas LLM pueden ser repentinos y de gran magnitud.

Este tema cubre las estrategias y herramientas para que los sistemas LLM escalen eficientemente: desde el autoscaling horizontal con Kubernetes y KEDA, hasta el caching de outputs para reducir la carga en el modelo, pasando por la gestión del pool de warm workers y las estrategias de priorización de tráfico.

Las dimensiones de la escalabilidad LLM

Escalabilidad horizontal: más réplicas del modelo

La escalabilidad horizontal añade más instancias del servidor de inferencia (más pods en Kubernetes) para manejar más tráfico concurrente. Es el mecanismo más directo: si un pod puede manejar 50 req/sec y necesitas 500 req/sec, necesitas 10 pods. Las limitaciones son el tiempo de cold start de cada nuevo pod (1-5 minutos), el coste de GPU (cada pod requiere una GPU), y la gestión del estado del KV cache (no se comparte entre pods sin infraestructura adicional).

Autoscaling con KEDA

HPA (Horizontal Pod Autoscaler) estándar de Kubernetes escala basándose en CPU o memoria, que no son los indicadores correctos para LLMs. KEDA (Kubernetes Event-Driven Autoscaler) permite escalar basándose en métricas personalizadas más relevantes: longitud de la cola de peticiones en espera, número de tokens en procesamiento, throughput de tokens generados por segundo, o utilización de GPU. KEDA puede escalar desde 0 (scale-to-zero para ahorrar costes en periodos de inactividad) hasta N pods según la demanda.

Warm pools: eliminar el cold start

El cold start de un pod de inferencia (el tiempo que tarda desde que Kubernetes decide crear un pod hasta que está listo para servir peticiones) es de 1-5 minutos para modelos grandes, inaceptable para sistemas con SLA de disponibilidad. La solución es mantener un pool de pods pre-calentados (warm pool): siempre hay N pods mínimos listos, aunque no estén procesando peticiones. El autoscaling escala hacia arriba añadiendo nuevos pods del pool warm (inmediatos) y hacia abajo terminando pods excedentes pero manteniendo el mínimo warm.

Estrategias avanzadas de escalabilidad

Semantic caching: evitar llamadas redundantes al modelo

El semantic caching almacena las respuestas a preguntas anteriores y las reutiliza cuando llega una pregunta semánticamente similar. A diferencia del caching exacto (que solo cachea consultas idénticas), el semantic caching usa embeddings para detectar consultas similares aunque no sean idénticas. GPTCache (open source) y Redis con soporte de vectores son las implementaciones más comunes. Para sistemas de QA con preguntas repetitivas (FAQ, documentación), el semantic caching puede reducir las llamadas al modelo en un 30-60%, reduciendo directamente el coste y la latencia.

Prioritización de tráfico y QoS

No todos los requests tienen la misma urgencia. Un sistema de priorización permite que las peticiones interactivas de usuario (que el usuario está esperando activamente) tengan prioridad sobre las peticiones batch (procesamiento de documentos en background). La implementación en Kubernetes usa Priority Classes (PriorityClass) para el scheduling de pods, y el Kubernetes Inference Gateway Extension soporta criticality-based routing para dirigir peticiones de alta prioridad a pods dedicados.

Request batching: maximizar el throughput del modelo

Los LLMs son más eficientes cuando procesan múltiples peticiones simultáneamente en un batch. vLLM implementa continuous batching automáticamente: agrupa dinámicamente las peticiones entrantes para maximizar la utilización de GPU. Para workloads batch (procesar miles de documentos), el batching explícito (enviar múltiples documentos en un solo request de /v1/chat/completions con system prompt compartido) puede mejorar el throughput 2-5x respecto a peticiones individuales.

SECCIÓN 4 · EJEMPLOS REALES

- Autoscaling con KEDA en sistema de soporte al cliente (picos en lunes mañana): el deployment de vLLM tiene min=2, max=20 réplicas. KEDA escala basándose en la longitud de la cola de requests pendientes (ScaledObject con una regla: si queue > 50, añade réplicas). El lunes a las 9am, la cola sube a 200 peticiones y KEDA escala a 8 réplicas en 5 minutos (los pods del warm pool están listos en 30 segundos, los nuevos pods tardan 3 minutos). El SLA se mantiene.
- Semantic caching en sistema de QA de RRHH (preguntas frecuentes): las 50 preguntas más frecuentes sobre políticas de empresa se hacen cientos de veces al día. GPTCache con redis-stack detecta preguntas semánticamente similares y devuelve la respuesta cacheada sin llamar al modelo. Hit rate del 65% en las primeras semanas. Reducción del 60% en llamadas al modelo, reducción del 55% en latencia media (responses cacheadas en <10ms vs 1-3s del modelo).
- Prioritización en sistema batch + interactivo (empresa de análisis): el mismo cluster de GPU sirve tanto el asistente interactivo de analistas (alta prioridad, baja latencia) como el procesamiento batch nocturno de informes (baja prioridad, puede esperar). Durante el día, las peticiones interactivas tienen prioridad con priority classes de Kubernetes y nunca se bloquean por el batch. Por la noche, el batch utiliza toda la capacidad disponible.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Escalar solo basándose en CPU/memoria con HPA estándar. La CPU y memoria de los pods de inferencia LLM no son buenos indicadores de la carga real. Un pod puede estar al 5% de CPU pero ser el cuello de botella de GPU al 100%. Usa KEDA con métricas de GPU (throughput de tokens, longitud de cola) para autoscaling efectivo.

Error 2: Semantic caching sin TTL (Time-To-Live). Las respuestas cacheadas pueden quedar obsoletas si la base de conocimiento cambia. Define siempre un TTL apropiado para el semantic cache: corto para información que cambia frecuentemente (noticias, precios), largo para información estable (documentación técnica, preguntas frecuentes sobre políticas que cambian raramente).

Error 3: No definir SLOs de escalabilidad antes de dimensionar. "El sistema debe escalar" no es un requisito de escalabilidad. Define: tiempo máximo de cold start (3 minutos), número máximo de usuarios concurrentes para el warm pool (50), tiempo de scale-up desde 50% a 100% de demanda (5 minutos). Estos SLOs definen la configuración del warm pool y el autoscaler.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Configurar KEDA para escalado de vLLM: instala KEDA con Helm (`helm install keda keda/keda`). Crea un `ScaledObject` que referencia el Deployment de vLLM y define las triggers: una trigger de tipo prometheus que lee la métrica `vllm:queue_requests_total` (longitud de la cola de requests pendientes en vLLM expuesta en `/metrics`). Define `minReplicaCount=2`, `maxReplicaCount=20`, y la regla de scaling (si `queue > 10`, añade una réplica). El `cooldown period` debe ser de al menos 10 minutos para evitar thrashing (escalado/desescalado rápido).

Para implementar semantic caching con GPTCache: `pip install gptcache`. Inicializa el cache con un encoder de embeddings (`sentence-transformers/all-MiniLM-L6-v2`) y un backend de almacenamiento (Redis). Configura el threshold de similitud (0.85 es un buen punto de partida: respuestas para preguntas con similitud coseno > 0.85 se consideran iguales). Monitoriza el hit rate del cache y ajusta el threshold según la tasa de false positives (respuestas incorrectas servidas por el cache).

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M9-T2 (Arquitecturas de despliegue): el autoscaling es un componente de la arquitectura de despliegue.
- M8-T6 (vLLM y TGI): el continuous batching de vLLM es el mecanismo que maximiza el throughput a nivel del servidor.
- M8-T5 (Gestión de tokens y costes): el semantic caching reduce directamente el coste de inferencia.

SECCIÓN 8 · RESUMEN EJECUTIVO

La escalabilidad LLM requiere: autoscaling horizontal con KEDA (métricas de cola y throughput, no CPU/memoria), warm pools para eliminar cold starts (2-3 pods mínimos siempre calientes), scale-to-zero fuera del horario de uso para ahorrar 40-60% de coste, semantic caching con GPTCache para reducir 30-60% de llamadas al modelo en

sistemas con preguntas repetitivas, y priorización de tráfico (interactivo vs batch) con Kubernetes Priority Classes. Para dimensionar el warm pool: $\text{ceil}(\text{P99 concurrentes} / \text{throughput por pod}) + \text{buffer de scale-up}$. El TTL del semantic cache es tan importante como el threshold de similitud.